4

Event Handling in the React Way

The focus of this chapter is **event handling**. React has a unique approach to handling events: declaring event handlers in JSX. We'll get things started by looking at how event handlers for particular elements are declared in JSX. Then, we'll explore **inline** and **higher-order event handler** functions.

Afterward, you'll learn how React maps event handlers to DOM elements under the hood. Finally, you'll learn about the synthetic events that React passes to event handler functions and how they're pooled for performance purposes. Once you've completed this chapter, you'll be comfortable implementing event handlers in your React components. At that point, your applications come to life for your users because they are then able to interact with them.

The following topics are covered in this chapter:

- Declaring event handlers
- Declaring inline event handlers

- Binding handlers to elements
- Using synthetic event objects
- Understanding event pooling

Technical requirements

The code presented in this chapter can be found at the following link: **https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter04**

Declaring event handlers

The differentiating factor with event handling in React components is that it's declarative. Compare this with something such as **jQuery**, where you have to write imperative code that selects the relevant DOM elements and attaches event handler functions to them.

The advantage of the declarative approach to event handlers in JSX markup is that they're part of the UI structure. Not having to track down code that assigns event handlers is mentally liberating.

In this section, you'll write a basic event handler so that you can get a feel for the declarative event handling syntax found in React applications. Then, you'll learn how to use generic event handler functions.

Declaring handler functions

Let's take a look at a basic component that declares an event handler for the click event of an element:

```
function MyButton(props) {
  const clickHandler = () => {
    console.log("clicked");
  };
  return <button onClick={clickHandler}>{props.children}</button>;
}
```

The event handler `clickHandler` function is passed to the `onClick` property of the `<button>` element. By looking at this markup, you can see exactly which code will run when the button is clicked.

View the official React documentation for the full list of supported event property names at **https://react.dev/reference/react-dom/components/common**.

Next, let's take a look at how to respond to more than one type of event using different event handlers with the same element.

Multiple event handlers

What I really like about the declarative event handler syntax is that it's easy to read when there's more than one handler assigned to an element. Sometimes, for example, there are two or three handlers for an element. Imperative code is difficult to work with for a single event handler, let alone several of them. When an element needs more handlers, it's just another JSX attribute. This scales well from a code-maintainability perspective, as this example shows:

```
function MyInput() {
  const onChange = () => {
    console.log("changed");
  };
  const onBlur = () => {
    console.log("blured");
  };
  return <input onChange={onChange} onBlur={onBlur} />;
}
```

This `<input>` element could have several more event handlers and the code would be just as readable.

As you keep adding more event handlers to your components, you'll notice that a lot of them do the same thing. Next, you'll learn about inline event handler functions.

Declaring inline event handlers

The typical approach to assigning handler functions to JSX properties is to use a **named** function. However, sometimes, you might want to use an **inline** function, where the function is defined as part of the markup. This is done by assigning an arrow function directly to the event property in the JSX markup:

```
function MyButton(props) {
  return (
    <button onClick={(e) => console.log("clicked", e)}>
      {props.children}
    </button>
  );
}
```

The main use of inlining event handlers like this is when you have a **static parameter** value that you want to pass to another function. In this example, you're calling `console.log` with the clicked string. You could have set up a special function for this purpose outside of the JSX markup by creating a new function or by using a higher-order function. But then you would have to think of yet another name for yet another function. Inlining is just easier sometimes.

Next, you'll learn about how React binds handler functions to the underlying DOM elements in the browser.

Binding handlers to elements

When you assign an event handler function to an element in JSX, React doesn't actually attach an event listener to the underlying DOM element. Instead, it adds the function to an internal mapping of functions. There's a single event listener on the document for the page. As events bubble up through the DOM tree to the document, the React handler checks to see whether any components have matching handlers. The process is illustrated here:

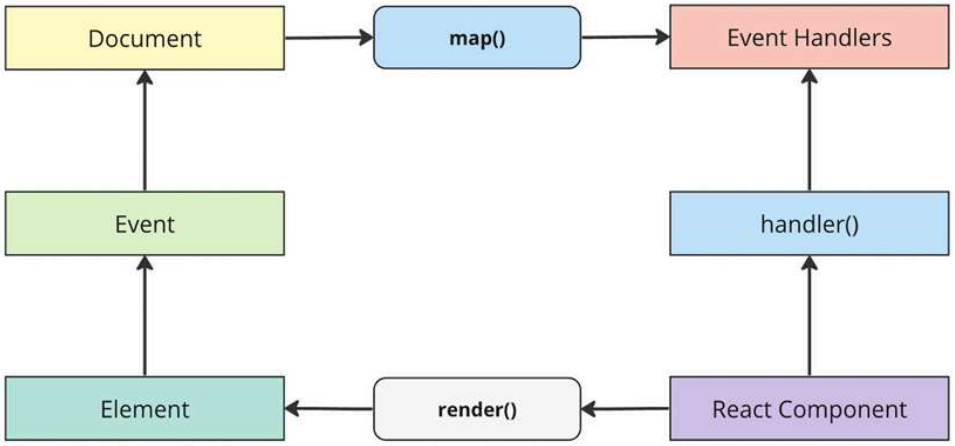

Figure 4.1: The event handler cycle

Why does React go through all of this trouble, you might ask? It's the same principle that I've been covering in the last few chapters: keep the declarative UI structures separated from the DOM as much as possible. The DOM is merely a render target; React's architecture allows it to remain agnostic about the final rendering destination and event system.

For example, when a new component is rendered, its event handler functions are simply added to the internal mapping maintained by React. When an event is triggered and it hits the document object, React maps the event to the handlers. If a match is found, it calls the handler. Finally, when the **React component** is removed, the handler is simply removed from the list of handlers.

None of these DOM operations actually touch the DOM. It's all abstracted by a single event listener. This is good for performance and the overall architecture (in other words, keeping the render target separate from the application code).

In the following section, you'll learn about the synthetic event implementation used by React to ensure good performance and safe asynchronous behavior.

Using synthetic event objects

When you attach an event handler function to a DOM element using the native `addEventListener` function, the callback will get an event argument passed to it. Event handler functions in React are also passed an event argument but it's not the standard event instance. It's called `SyntheticEvent` and it's a simple wrapper for native event instances.

Synthetic events serve two purposes in React:

- They provide a consistent event interface, normalizing browser inconsistencies.
- They contain information that's necessary for propagation to work.

Here's a diagram of the synthetic event in the context of a **React component**:

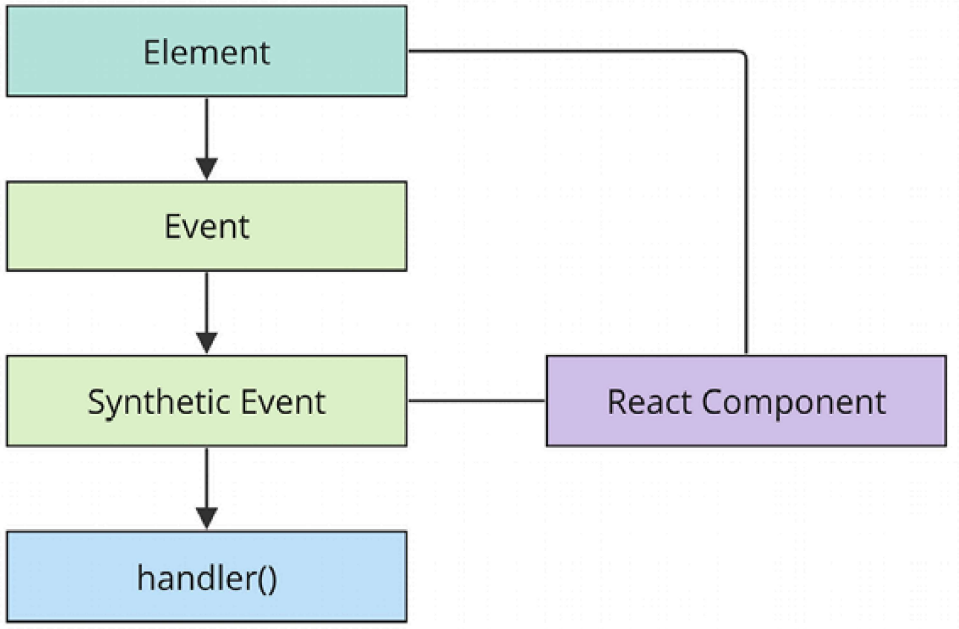

Figure 4.2: How synthetic events are created and processed

When a DOM element that is part of a **React component** dispatches an event, React will handle the event because it sets up its own listeners for them. Then, it will either create a new **synthetic event** or reuse one from the pool, depending on availability. If there are any event handlers declared for the component that match the DOM event that was dispatched, they will run with the synthetic event passed to them.

The event object in React has properties and methods similar to those in native JavaScript events. You can access properties such as `event.target` to retrieve the DOM element that trig-

gered the event, or `event.currentTarget` to refer to the element to which the event handler is attached.

Additionally, the event object provides methods like `event.preventDefault()` to prevent the default behavior associated with the event, such as form submissions or link clicks. You can also use `event.stopPropagation()` to stop the event from propagating further up the component tree, preventing event bubbling.

Event propagation works differently in React compared to traditional JavaScript event handling. In the traditional approach, events typically bubble up through the DOM tree, triggering handlers on ancestor elements.

In React, event propagation is based on the component hierarchy rather than the DOM hierarchy. When an event occurs in a child component, React captures the event at the root of the component tree and then traverses down to the specific component that triggered the event. This approach, known as event delegation, simplifies event handling by centralizing the event logic at the root of the component tree.

React's event delegation provides several benefits. First, it reduces the number of event listeners attached to individual DOM elements, resulting in improved performance. Second, it allows you to handle events for dynamically created or re-

moved elements without worrying about attaching or detaching event listeners manually.

In the next section, you'll see how these synthetic events are pooled for performance reasons and the implications of this on asynchronous code.

Understanding event pooling

One challenge of wrapping native event instances is that it can cause performance issues. Every synthetic event wrapper that's created will also need to be garbage collected at some point, which can be expensive in terms of CPU time.

When the garbage collector is running, none of your JavaScript code is able to run. This is why it's important to be memory-efficient; frequent garbage collection means less CPU time for code that responds to user interactions.

For example, if your application only handles a few events, this wouldn't matter much. But even by modest standards, applications respond to many events, even if the handlers don't actually do anything with them. This is problematic if React constantly has to allocate new synthetic event instances.

React deals with this problem by allocating a **synthetic instance pool**. Whenever an event is triggered, it takes an instance from the pool and populates its properties. When the event handler has finished running, the **synthetic event** instance is released back into the pool, as shown here:

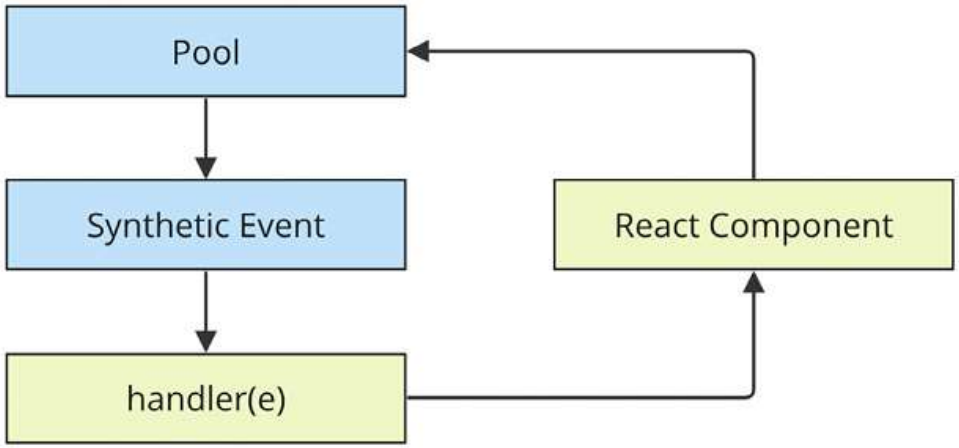

Figure 4.3: Synthetic events are reused to save memory resources

This prevents the garbage collector from running frequently when a lot of events are triggered. The pool keeps a reference to the synthetic event instances, so they're never eligible for garbage collection. React never has to allocate new instances either.

However, there is one gotcha that you need to be aware of. It involves accessing the synthetic event instances from asynchronous code in your event handlers. This is an issue because, as soon as the handler has finished running, the instance goes back into the pool. When it goes back into the pool, all of its properties are cleared.

Here's an example that shows how this can go wrong:

```
function fetchData() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve();
    }, 1000);
  });
}
function MyButton(props) {
  function onClick(e) {
    console.log("clicked", e.currentTarget.style);
    fetchData().then(() => {
      console.log("callback", e.currentTarget.style);
    });
  }
  return <button onClick={onClick}>{props.children}</button>;
}
```

The second call to `console.log` is attempting to access a synthetic event property from an asynchronous callback that

doesn't run until the event handler completes, which causes the event to empty its properties. This results in a warning and an undefined value.

The aim of this example is to illustrate how things can break when you write asynchronous code that interacts with events. Just don't do it!

In this section, you learned that events are pooled for performance reasons, which means that you should never access event objects in an asynchronous way.

Summary

This chapter introduced you to event handling in React. The key differentiator between React and other approaches to event handling is that handlers are declared in JSX markup. This makes tracking down which elements handle which events much simpler.

You learned that having multiple event handlers on a single element is a matter of adding new JSX properties. Then, you learned about inline event handler functions and their potential use, as well as how React actually binds a single DOM event handler to the document object.

Synthetic events are abstractions that wrap native events; you learned why they're necessary and how they're pooled for efficient memory consumption.

In the next chapter, you'll learn how to create components that are reusable for a variety of purposes. Instead of writing new components for each use case that you encounter, you'll learn the skills necessary to refactor existing components so that they can be used in more than one context.